Fractal 구현 분석

개요

Fractal 프로토콜의 구현은 `fractal_iop_parameters` 클래스와 `matrix_indexer` 클래스를 중심으로 구성됩니다. 이 문서에서는 libiop 라이브러리에서 Fractal이 어떻게 구현되어 있는지, 특히 인덱스 도메인 설정, 전처리 과정, 그리고 행렬 도메인과 코드워드 도메인의 분리와 최적화에 대해 상세히 분석합니다.

fractal_iop_parameters 클래스 분석

클래스 구조와 핵심 멤버 변수

```
template<typename FieldT>
class fractal_iop_parameters {
protected:
size_t security_parameter_; // 보안 매개변수
size_t pow_bits_; // 작업 증명 비트 수
size_t RS_extra_dimensions_; // Reed-Solomon 추가 차원
bool make_zk_; // 영지식 모드 활성화 여부
// 제약 시스템
std::shared_ptr<r1cs_constraint_system<FieldT>> constraint_system_;
// 도메인 설정
field_subset<FieldT> index_domain_; // 인덱스 도메인 K
```

```
field_subset<FieldT> matrix_domain_; // 행렬 도메인 H

field_subset<FieldT> codeword_domain_; // 코드워드 도메인 L

size_t codeword_domain_dim_;

size_t query_bound_;

// 하위 프로토콜 매개변수들

LDT_instance_reducer_params<FieldT> LDT_reducer_params_;

FRI_protocol_parameters<FieldT> FRI_params_;

encoded_aurora_parameters<FieldT> encoded_aurora_params_;

};
```

매개변수 초기화 과정

1. 제약 시스템 검증

```
fractal_iop_parameters<FieldT>::fractal_iop_parameters(

const  size_t security_parameter,

const  size_t pow_bits,

const  size_t RS_extra_dimensions,

const  bool make_zk,

const std::shared_ptr<r1cs_constraint_system<FieldT>> &constraint_system)

{

// 제약 조건 수가 2의 거듭제곱인지 확인

if (!libff::is_power_of_2(this->constraint_system_->num_constraints()))

{
```

```
throw std::invalid_argument(
"Fractal requires the number of constraints to be a power of two");
}
// 행렬이 정사각형인지 확인 (홀로그래픽 IOP 요구사항)
if (this->constraint_system_->num_constraints() !=
this->constraint_system_->num_variables() + 1)
{
throw std::invalid_argument(
"Fractal requires the matrices to be square");
}
}
```

이러한 제약 조건들은 홀로그래픽 IOP의 효율적인 구현을 위해 필요합니다:

- **2의 거듭제곱 조건**: FFT 기반 다항식 연산의 효율성
- **정사각 행렬 조건**: 행렬 인덱싱과 도메인 설정의 일관성

2. 도메인 크기 계산

```
// 각 행렬 A, B, C의 최대 비영 원소 개수 계산
const  size_t max_num_nonzero_entries_per_matrix = std::max({
r1cs_sparse_matrix<FieldT>(this->constraint_system_, r1cs_sparse_matrix_A).num_n
r1cs_sparse_matrix<FieldT>(this->constraint_system_, r1cs_sparse_matrix_B).num_n
r1cs_sparse_matrix<FieldT>(this->constraint_system_, r1cs_sparse_matrix_C).num_n
});
```

```
// 인덱스 도메인 크기 설정
const  size_t index_domain_dim = libff::log2(max_num_nonzero_entries_per_matrix)
this->index_domain_ = field_subset<FieldT>(1ull << index_domain_dim);


// 행렬 도메인 설정
this->matrix_domain_ = field_subset<FieldT>(this->constraint_system_->num_constr


3. 코드워드 도메인 설정


// 최대 테스트 차수 계산 (현재는 휴리스틱 값 사용)
const  size_t max_tested_degree = 4 * this->index_domain_.num_elements();


// 코드워드 도메인 차원 계산
this->codeword_domain_dim_ = libff::log2(max_tested_degree) + this->RS_extra_dim


// 시프트된 코드워드 도메인 생성
const field_subset<FieldT> unshifted_codeword_domain(1ull << this->codeword_doma
const FieldT codeword_domain_shift = unshifted_codeword_domain.element_outside_o
this->codeword_domain_ = field_subset<FieldT>(
1ull << this->codeword_domain_dim_, codeword_domain_shift);
```

인덱스 도메인 설정

도메인 간의 관계

```
|K| ≥ max(|A|₀, |B|₀, |C|₀) (인덱스 도메인)

|H| = 제약 조건 수 = 변수 수 + 1 (행렬 도메인)

|L| = 2^(log₂ (max_tested_degree) + RS_extra_dimensions) (코드워드 도메인)
```

여기서:

- `|A|₀, |B|₀, |C|₀` : 각각 행렬 A, B, C의 비영 원소 개수
- `max_tested_degree` : 테스트되는 다항식의 최대 차수
- `RS_extra_dimensions` : Reed-Solomon 코드의 추가 차원

도메인 최적화 전략

1. **인덱스 도메인 최적화**

- 비영 원소 개수에 맞춰 최소 크기로 설정
- 2의 거듭제곱으로 반올림하여 FFT 효율성 확보

2. **코드워드 도메인 최적화**

- Reed-Solomon 코드의 거리 특성 고려
- 보안 매개변수에 따른 추가 차원 설정

fractal_indexer 전처리 과정

matrix_indexer 클래스 구조

```
template<typename FieldT>
class matrix_indexer {
```

```
protected:

iop_protocol<FieldT> &IOP_;

// 도메인 핸들들

domain_handle index_domain_handle_; // K

domain_handle matrix_domain_handle_; // H

domain_handle codeword_domain_handle_; // L

size_t input_variable_dim_;

std::shared_ptr<sparse_matrix<FieldT>> matrix_;

// 실제 도메인 객체들

field_subset<FieldT> index_domain_;

field_subset<FieldT> matrix_domain_;

field_subset<FieldT> codeword_domain_;

// 오라클 핸들들

oracle_handle row_oracle_handle_;

oracle_handle col_oracle_handle_;

oracle_handle val_oracle_handle_;

oracle_handle row_times_col_oracle_handle_;

// 이변수 라그랑주 다항식

bivariate_lagrange_polynomial<FieldT> bivariate_lagrange_poly_;

};
```

전처리 과정의 핵심 단계

1. 오라클 등록

```
void matrix_indexer<FieldT>::register_oracles()
{
const  size_t oracle_degree_bound = this->index_domain_.num_elements();
assert(oracle_degree_bound >= this->matrix_->num_nonzero_entries());


this->row_oracle_handle_ =
this->IOP_.register_index_oracle(this->codeword_domain_handle_, oracle_degree_bo
this->col_oracle_handle_ =
this->IOP_.register_index_oracle(this->codeword_domain_handle_, oracle_degree_bo
this->val_oracle_handle_ =
this->IOP_.register_index_oracle(this->codeword_domain_handle_, oracle_degree_bo
this->row_times_col_oracle_handle_ =
this->IOP_.register_index_oracle(this->codeword_domain_handle_, oracle_degree_bo
}
```

2. 행렬 분해와 인덱싱

```
std::vector<std::vector<FieldT>> matrix_indexer<FieldT>::compute_oracles_over_K(
{
// 이변수 라그랑주 다항식 초기화
this->bivariate_lagrange_poly_ =
bivariate_lagrange_polynomial<FieldT>(this->matrix_domain_);
// 각 오라클의 평가값 저장
```

```
std::vector<FieldT> row_evals, col_evals, val_evals, row_times_col_evals;

std::size_t num_nonzero_cnt = 0;

for (size_t i = 0; i < this->matrix_->num_rows(); i++)

{

const linear_combination<FieldT> row = this->matrix_->get_row(i);

const FieldT row_index_elem = this->matrix_domain_.element_by_index(i);



for (auto &term : row.terms)

{

num_nonzero_cnt++;

// 행 인덱스 저장

row_evals.emplace_back(row_index_elem);

// 열 인덱스 계산 및 저장

const  size_t col_index =

this->matrix_domain_.reindex_by_subset(this->input_variable_dim_, term.index_);

const FieldT col_index_elem = this->matrix_domain_.element_by_index(col_index);

col_evals.emplace_back(col_index_elem);

// 행×열 값 저장

row_times_col_evals.emplace_back(row_index_elem * col_index_elem);

// 값 계산 (이변수 라그랑주 다항식 고려)

const FieldT col_derivative =

this->bivariate_lagrange_poly_.evaluation_at_point(col_index_elem, col_index_ele

const FieldT val_eval = term.coeff_ * col_derivative.inverse();
```

```
val_evals.emplace_back(val_eval);
}
}
// 나머지 위치를 0으로 패딩
const FieldT arbitrary_elem_in_H = this->matrix_domain_.element_by_index(0);
for (std::size_t i = num_nonzero_cnt; i < this->index_domain_.num_elements(); i+
{
row_evals.emplace_back(arbitrary_elem_in_H);
col_evals.emplace_back(arbitrary_elem_in_H);
val_evals.emplace_back(FieldT::zero());
}
// 전치 행렬을 위해 행과 열 교환
row_evals.swap(col_evals);
return {row_evals, col_evals, val_evals, row_times_col_evals};
}
```

3. 다항식 변환과 제출

```
void matrix_indexer<FieldT>::compute_oracles()
{
std::vector<std::vector<FieldT>> index_oracles_over_K = this->compute_oracles_ov
// 인덱스 도메인에서 코드워드 도메인으로 변환
const std::vector<FieldT> row_poly_over_codeword_domain =
```

```
FFT_over_field_subset<FieldT>(

IFFT_over_field_subset<FieldT>(index_oracles_over_K[0], this->index_domain_),

this->codeword_domain_);

this->IOP_.submit_oracle(this->row_oracle_handle_, row_poly_over_codeword_domain

// 다른 오라클들도 동일한 방식으로 처리

// ...

}
```

전처리 과정의 데이터 구조

1. 희소 행렬 표현

```
R1CS 행렬 M → (row, col, val) 다항식들

각 비영 원소 M[i,j] = v에 대해:
```

- `row(k) = i` (행 인덱스)
- `col(k) = j` (열 인덱스)
- `val(k) = v / u_H(j,j)` (정규화된 값)

여기서 k는 인덱스 도메인 K의 원소이고, u_H는 이변수 라그랑주 다항식입니다.

2. 홀로그래픽 표현

```
M(x,y) = Σ_{k∈K} val(k) · u_H(row(k), x) · u_H(col(k), y)
```

이 표현을 통해 행렬 M의 임의의 위치 (x,y)에서의 값을 효율적으로 계산할 수 있습니다.

행렬 도메인과 코드워드 도메인의 분리

도메인 분리의 목적

1. 기능적 분리

- **행렬 도메인 H**: 제약 조건과 변수의 식별
- **코드워드 도메인 L**: Reed-Solomon 인코딩과 오류 정정
- **인덱스 도메인 K**: 행렬의 비영 원소 인덱싱

2. 성능 최적화

```
// 행렬 도메인: 제약 시스템 크기에 맞춤
this->matrix_domain_ = field_subset<FieldT>(this->constraint_system_->num_constr

// 코드워드 도메인: Reed-Solomon 코드 요구사항에 맞춤
this->codeword_domain_dim_ = libff::log2(max_tested_degree) + this->RS_extra_dim
```

최적화 기법

1. 도메인 크기 최적화

```
|H| = n (제약 조건 수)
|K| ≥ max(|A|₀, |B|₀, |C|₀) (최소 필요 크기)
|L| = 2^(⌈log₂ (max_tested_degree)⌉ + RS_extra_dimensions)
```

2. 메모리 효율성

- 각 도메인별로 필요한 최소 크기만 할당
- 도메인 간 변환 시 임시 메모리 사용 최소화
- 스트리밍 방식의 다항식 변환

3. 계산 복잡도 최적화

```
// FFT 기반 도메인 변환
const std::vector<FieldT> poly_over_codeword_domain =
FFT_over_field_subset<FieldT>(
IFFT_over_field_subset<FieldT>(evaluations_over_index_domain, this->index_domain
this->codeword_domain_);
```

시간 복잡도:

- 인덱스 도메인에서 계수 형태로 변환: O(|K| log |K|)
- 계수에서 코드워드 도메인으로 변환: O(|L| log |L|)
- 전체: O(|K| log |K| + |L| log |L|)

보안성과 효율성의 균형

1. Reed-Solomon 추가 차원

```
const  size_t RS_extra_dimensions = 3; // 일반적인 설정값
```

- 추가 차원이 클수록: 보안성 향상, 하지만 증명 크기 증가
- 추가 차원이 작을수록: 효율성 향상, 하지만 보안성 감소

2. 지역화 매개변수 최적화

```
// FRI 프로토콜의 지역화 매개변수 배열
std::vector<size_t> localization_parameter_array =
compute_argument_size_optimal_localization_parameters<FieldT>(
oracle_locality_vector,
codeword_dim,
num_query_sets,
interactive_repetitions,
max_tested_degree,
hash_size);
```

3. 쿼리 복잡도 최적화

- 보안 매개변수에 따른 쿼리 수 조정
- 작업 증명(PoW)을 통한 쿼리 사운드니스 부스팅
- 상호작용 라운드 수와 쿼리 수의 트레이드오프

구현상의 주요 특징

1. 템플릿 기반 설계

```
template<typename FieldT>
class fractal_iop_parameters;

template<typename FieldT>
```

```
class matrix_indexer;
```

다양한 유한체에서 동작 가능한 일반적인 구현을 제공합니다.

2. 모듈화된 구조

- 각 구성 요소가 독립적으로 테스트 가능
- 매개변수 변경이 다른 모듈에 미치는 영향 최소화
- 확장성과 유지보수성 향상

3. 성능 프로파일링 지원

```
libff::enter_block("Fractal indexing");

// 인덱싱 작업 수행

libff::leave_block("Fractal indexing");
```

실제 성능 측정과 최적화를 위한 프로파일링 기능이 내장되어 있습니다.

이러한 구현 분석을 통해 Fractal 프로토콜이 어떻게 홀로그래픽 IOP의 이론적 장점을 실제 코드로 구현하고 있는지 이해할 수 있습니다.